Week 3 - CloudFormation StackSets (self-managed vs service-managed, Organizations integration, automatic deployment)

DOP-C02 Domain 2 (Configuration Management and IaC). You must:

1. **Know when to use StackSets**: one template deployed to **many accounts x many Regions**.
2. **Choose the permission model**: **self-managed** vs **service-managed** (AWS Organizations).
3. **Set up Organizations integration**: trusted access, delegated administrator, OU targets, account filters.
4. **Use automatic deployment** for new and moved accounts.
5. **Control the rollout**: Region order, concurrency, failure tolerance, managed execution.

Exam pattern: "deploy the same thing everywhere" = **StackSets**; "every new account gets it automatically" = **service-managed + auto-deployment**; "why did it fail or skip?" = **roles, management account, failure tolerance, Regions**.

Part 1 - Core concepts

Term	Meaning
StackSet	Template + parameters + settings, kept in the **administrator account** in one home Region
Stack instance	A **pointer** to one stack in one target account and Region (exists even if the stack failed)
Stack	The real CloudFormation stack in the target account and Region
Administrator account	Where you create and manage the StackSet (management account or **delegated admin**)
Target accounts	Where the stacks are created
Operation	Create / update / delete / add or remove instances, with an operation ID and per-instance results

```
Admin account (home Region, e.g. us-east-1)
   StackSet "security-baseline"
      instance: 111111111111 / us-east-1  -> stack
      instance: 111111111111 / eu-west-1  -> stack
      instance: 222222222222 / us-east-1  -> stack
      instance: 222222222222 / eu-west-1  -> stack
```

Typical uses:

- Org-wide **security baseline**: AWS Config recorder and rules, CloudTrail, GuardDuty or Security Hub settings.
- **IAM roles** in every account (cross-account access, break-glass, audit).
- Log shipping, budgets, alarms, standard SNS topics.
- Rolling out **Registry extensions, macros and Hooks** to every account and Region.
- **AWS Control Tower** uses StackSets internally, including Customizations for Control Tower (CfCT).

Part 2 - Permission models

Self-managed permissions

You create the IAM roles yourself:

Role	Where	Trusts	Purpose
AWSCloudFormationStackSetAdministrationRole	Admin account	`cloudformation.amazonaws.com`	**Assumes the execution role** in targets
AWSCloudFormationStackSetExecutionRole	**Every** target account	The **admin account**	Creates the stack resources

- Targets are **account IDs**: any account, even **outside your Organization**, including the **admin account itself**.
- **No automatic deployment**: for each new account, create the execution role there, then run `create-stack-instances`.
- **Custom role names per StackSet** (`--administration-role-arn`, `--execution-role-name`) give **least privilege**. Example: one StackSet may only deploy Config resources, and only some admins may pass that admin role.
- Typical failure: execution role missing, wrong trust, or not enough permissions. The instance ends **INOPERABLE** or FAILED with a message that the account should have `AWSCloudFormationStackSetExecutionRole`.

Service-managed permissions (AWS Organizations)

- Turn on **trusted access** between CloudFormation StackSets and AWS Organizations (console: StackSets, Activate trusted access).
- CloudFormation creates **service-linked roles** automatically: `AWSServiceRoleForCloudFormationStackSetsOrgAdmin` in the management account, and `AWSServiceRoleForCloudFormationStackSetsOrgMember` plus a `stacksets-exec-*` execution role in member accounts.
- Targets are **OUs** or the **organization root**, optionally narrowed with account filters.
- Supports **automatic deployment** to new and moved accounts.
- Run it from the **management account** or a **delegated administrator** account.
- **It never deploys to the management account**, even when you target the root. Use a self-managed StackSet or a normal stack for the management account.

Side by side

	Self-managed	Service-managed
IAM roles	**You** create admin and execution roles	**Automatic** service-linked roles
Needs Organizations	No	**Yes**, trusted access on
Target by	Account IDs	**OUs** / root (+ account filters)
Accounts outside the org	YES	NO
Management account as target	YES	**NO, never**
Automatic deployment	NO	**YES**
Delegated administrator	NO	**YES**
Custom least-privilege roles per StackSet	YES	NO (managed roles)
Exam keywords	Outside the org, custom roles, include management account	Every new account, OU, Organizations, no manual roles

Part 3 - Organizations integration

Trusted access

- Enable it from the **CloudFormation console** in the management account (recommended; it also creates the service-linked roles).
- If trusted access is **disabled**, service-managed StackSets stop working.

Delegated administrator

Register a member account (often a security tooling or infrastructure account) from the management account:

```
aws organizations register-delegated-administrator \
  --service-principal member.org.stacksets.cloudformation.amazonaws.com \
  --account-id 333333333333
```

- Why: **keep people out of the management account** (least use of the management account is a best practice).
- **Every call from the delegated admin must pass** `--call-as DELEGATED_ADMIN`**.** The default is `SELF`, so without it you get "StackSet not found" or permission errors. The console has the same choice.
- Up to **5** delegated administrators for StackSets per organization.
- A delegated admin manages **service-managed** StackSets only and still **can't target the management account**.

Deployment targets and account filters (service-managed)

`DeploymentTargets` takes `OrganizationalUnitIds`, `Accounts` and `AccountFilterType`.

AccountFilterType	Effect
`NONE` (default)	All accounts in the OUs
`INTERSECTION`	Only the listed accounts that are **inside** the OUs
`DIFFERENCE`	All OU accounts **except** the listed ones (skip a sandbox)
`UNION`	OU accounts **plus** the listed accounts

Targeting an OU **includes its child OUs**.

Part 4 - Automatic deployment (service-managed only)

```
aws cloudformation create-stack-set \
  --stack-set-name security-baseline \
  --template-url https://s3.amazonaws.com/bucket/baseline.yaml \
  --permission-model SERVICE_MANAGED \
  --auto-deployment Enabled=true,RetainStacksOnAccountRemoval=false \
  --capabilities CAPABILITY_NAMED_IAM
```

Event	What happens
New account **created in or moved into** a targeted OU	Instances **created automatically**, in the **Regions already on the StackSet**
Account **leaves** the targeted OUs (moved out or removed from org)	Instances **deleted** (`RetainStacksOnAccountRemoval=false`) or **kept** as plain stacks (`true`)
Account moves **between two targeted OUs**	Nothing is redeployed; still a target
You **add a new Region** later	**Not automatic**: run `create-stack-instances` for the OUs with that Region

- Auto-deployment is set **on the StackSet**, not per OU, and applies to all its target OUs.
- It only reacts to **Organizations membership changes**. It doesn't add Regions or repair failed stacks.
- `RetainStacksOnAccountRemoval=true` keeps resources (for example a logging role) when an account leaves; the stack is then no longer managed by the StackSet.
- Classic pairing: "every new account must automatically get CloudTrail / Config / an IAM role" = **service-managed StackSet targeting the OU with auto-deployment** (often with **Control Tower Account Factory**).

Part 5 - Operation preferences (rollout control)

Setting	Meaning
RegionOrder	The order Regions are deployed in (test Region first)
RegionConcurrencyType	`SEQUENTIAL` (default, one Region at a time) or `PARALLEL`
MaxConcurrentCount / Percentage	Accounts **per Region** deployed at the same time
FailureToleranceCount / Percentage	Account failures **per Region** allowed before CloudFormation **stops in that Region and doesn't continue to other Regions**
ConcurrencyMode	`STRICT_FAILURE_TOLERANCE` (default, lowers concurrency to stay within tolerance + 1) or `SOFT_FAILURE_TOLERANCE` (keeps concurrency, faster)

- Failure tolerance is **per Region**.
- In strict mode, max concurrent is **at most failure tolerance + 1**.
- **Failure tolerance 0** = the first failure stops everything (safest).
- Percentages are **rounded down** (concurrency is at least 1).
- Canary-style rollout: **RegionOrder** with a test Region first, **SEQUENTIAL**, low concurrency, tolerance 0.
- Fast rollout to hundreds of accounts accepting a few failures: **higher MaxConcurrentPercentage**, **FailureTolerancePercentage**, **SOFT_FAILURE_TOLERANCE**.

Managed execution

- Without it, StackSet operations run **one at a time**; a second one fails with `OperationInProgressException`.
- `--managed-execution Active=true`: CloudFormation runs **non-conflicting operations concurrently** and **queues conflicting ones**.
- Exam: "pipeline updates to a StackSet fail because another operation is in progress" = **managed execution**.

Part 6 - Updates, parameters and instance lifecycle

Action	CLI
Create the StackSet (template only, no stacks yet)	`create-stack-set`
Add accounts and Regions	`create-stack-instances --accounts` or `--deployment-targets`, `--regions`
Change template or parameters for all instances	`update-stack-set` (can be limited to some accounts / Regions)
Per-instance parameter overrides	`create-stack-instances` or `update-stack-instances --parameter-overrides`
Remove stacks	`delete-stack-instances` (`--retain-stacks` keeps the stack, it just leaves the set)
Delete the StackSet	`delete-stack-set`, **only after all instances are removed**
Bring existing stacks into a StackSet	`import-stacks-to-stack-set`
Find manual changes	`detect-stack-set-drift`
Check results	`list-stack-set-operations`, `list-stack-set-operation-results`, `list-stack-instances`

Stack instance status

Status	Meaning
CURRENT	Matches the latest StackSet version
OUTDATED	Last operation failed or stopped for this instance, or wasn't applied yet
INOPERABLE	A delete failed and the instance is out of sync; usually `delete-stack-instances --retain-stacks`, then manual cleanup

Detailed statuses: SUCCEEDED, FAILED, CANCELLED, PENDING, RUNNING, **SKIPPED_SUSPENDED_ACCOUNT**, FAILED_IMPORT.

Other details

- **Capabilities**: IAM resources need `CAPABILITY_NAMED_IAM` / `CAPABILITY_IAM`; transforms need `CAPABILITY_AUTO_EXPAND`.
- **Global resources**: IAM roles are **global** with fixed names. Deploying an IAM role StackSet to several Regions fails with **"already exists"** in the second Region. Deploy IAM to **one Region only** (separate StackSet) or include the Region in the name.
- **Opt-in Regions** must be **enabled** in the target account.
- **Suspended accounts** are skipped (SKIPPED_SUSPENDED_ACCOUNT).
- Stacks in target accounts are **normal stacks**: a local admin can change them (drift). Protect them with **SCPs** that deny changes to baseline resources and `stacksets-exec-*` roles.
- **StackSets as code**: the `AWS::CloudFormation::StackSet` resource, often deployed from a pipeline in the delegated admin account.
- **CodePipeline actions**: `CloudFormationStackSet` (create or update the StackSet and instances) and `CloudFormationStackInstances` (add instances, per-instance parameters), using the operation preferences for waves.

Part 7 - StackSets vs other multi-account tools

Requirement	Best answer
Same CFN resources in many accounts and Regions, managed centrally	**StackSets**
New accounts get the baseline automatically	**Service-managed StackSets + auto-deployment** (or Control Tower + CfCT)
Users choose and launch approved products themselves	**Service Catalog**
Prevent actions (guardrails)	**SCPs** / Control Tower preventive controls
Org-wide Config rules	**Config conformance packs / org rules** (StackSets also works)
Org-wide WAF, security group, Shield policies	**Firewall Manager**
One application pipeline to dev, test, prod accounts with approvals	**CodePipeline with cross-account roles**
Create new accounts with a baseline	**Control Tower Account Factory** (AFT for Terraform)

Part 8 - Exam traps

Trap	Truth
Service-managed StackSet also covers the management account	**Never.** Use self-managed or a normal stack
Self-managed with auto-deployment	Auto-deployment is **service-managed only**
Service-managed to accounts outside the org	Not possible; **self-managed** with execution roles
Delegated admin gets "StackSet not found"	Missing `--call-as DELEGATED_ADMIN`
Auto-deployment adds my new Region	No; it reacts to **account moves into OUs**. Add Regions with `create-stack-instances`
Self-managed fails in a new account	**Execution role missing** or doesn't trust the admin account
Delete the StackSet while instances exist	Fails; **delete instances first** (or retain stacks)
Keep resources when an account leaves	**RetainStacksOnAccountRemoval=true** / `--retain-stacks`
IAM role StackSet fails in the 2nd Region	Global name conflict; **deploy to one Region**
Stop after the first failure	**FailureToleranceCount=0** (per Region)
Operation fails because another is running	**Managed execution Active=true**
Faster rollout despite some failures	**SOFT_FAILURE_TOLERANCE** + higher concurrency
Instance shows OUTDATED	Last operation failed or stopped for it; fix and **re-run**
INOPERABLE instance	`delete-stack-instances --retain-stacks`, clean up
Local admins change baseline resources	Detect with **StackSet drift detection**, prevent with **SCP**
Let users pick products themselves	**Service Catalog**, not StackSets
Exclude one account from an OU	**AccountFilterType DIFFERENCE**

Part 9 - Hands-on lab (about 45 minutes, very low cost)

Labs 1 to 3 need **one account** (self-managed can target the admin account itself). Lab 4 needs an **Organization** with one member account.

Lab 1 - Self-managed roles

1. Deploy the AWS sample templates `AWSCloudFormationStackSetAdministrationRole.yml` and `AWSCloudFormationStackSetExecutionRole.yml` (StackSets docs, "Grant self-managed permissions"). For the execution role parameter `AdministratorAccountId`, use **your own account ID**.
2. Check the trust policies: admin role trusts `cloudformation.amazonaws.com`; execution role trusts `arn:aws:iam::<your-id>:root`.

Lab 2 - Multi-Region StackSet with operation preferences

1. Create `baseline.yaml`:

```
Parameters:
  Env: { Type: String, Default: dev }
Resources:
  AlertsTopic:
    Type: AWS::SNS::Topic
    Properties:
      TopicName: !Sub "baseline-alerts-${Env}-${AWS::Region}"
Outputs:
  TopicArn: { Value: !Ref AlertsTopic }
```

2. Create the StackSet and instances:

```
aws cloudformation create-stack-set --stack-set-name lab-baseline \
  --template-body file://baseline.yaml --permission-model SELF_MANAGED
aws cloudformation create-stack-instances --stack-set-name lab-baseline \
  --accounts <your-id> --regions us-east-1 us-west-2 eu-west-1 \
  --operation-preferences RegionConcurrencyType=SEQUENTIAL,RegionOrder=us-west-2,us-east-1,eu-west-1,Failure
    ToleranceCount=0,MaxConcurrentCount=1
```

3. Watch `list-stack-set-operations` and `list-stack-instances`. Confirm the Region order and that instances become **CURRENT**.

Lab 3 - Overrides, failure, drift, cleanup

1. **Override**: `update-stack-instances --accounts <id> --regions eu-west-1 --parameter-overrides ParameterKey=Env,ParameterValue=prod`. Only eu-west-1 changes.
2. **Failure**: add an IAM role with a fixed `RoleName` and run `update-stack-set --capabilities CAPABILITY_NAMED_IAM`. The first Region succeeds, the next fails with **"already exists"**, the operation **stops** (tolerance 0) and remaining instances are **OUTDATED**. Remove the fixed name and update again.
3. **Drift**: change the SNS topic display name by hand in one Region, run `detect-stack-set-drift`, and see that instance **DRIFTED**.
4. **Cleanup**: try `delete-stack-set` first and see it **fail**. Then `delete-stack-instances --regions ... --no-retain-stacks`, then `delete-stack-set`.

Lab 4 - Service-managed with auto-deployment (Organizations)

1. In the management account, CloudFormation, StackSets: **activate trusted access**. Optionally register a member as **delegated admin**.
2. Create a **SERVICE_MANAGED** StackSet targeting a test OU, **auto-deployment on**, `RetainStacksOnAccountRemoval=false`, one Region. From the delegated admin add `--call-as DELEGATED_ADMIN`.
3. **Move an account** into the OU: an operation starts **by itself**. Move it out: the instance is **deleted**.
4. Target the **root** and confirm the **management account gets no instance**.

Part 10 - Check yourself

Questions

1. Which two roles do self-managed StackSets need, and in which accounts?
2. Which permission model can deploy to accounts outside your Organization?
3. Every new account in the Workloads OU must get a Config recorder with no manual steps. What do you set up?
4. A service-managed StackSet targets the root, but the management account has no stack. Why, and what's the fix?
5. A delegated admin's CLI calls fail with "StackSet not found". What's missing?
6. Why use a delegated administrator?
7. An account moves out of a targeted OU. What decides what happens to its stack?
8. You add ap-south-1 to the baseline. Does auto-deployment add it to existing accounts?
9. How do you deploy to all accounts in an OU except one sandbox account?
10. Which settings stop the rollout at the first failed account and deploy one Region at a time?
11. Pipeline updates to a StackSet fail with OperationInProgressException. Fix?
12. An IAM role StackSet succeeds in us-east-1 but fails in eu-west-1 with "already exists". Why?
13. `delete-stack-set` fails. What must you do first?
14. How do you clean up an INOPERABLE instance?
15. Local admins keep changing baseline resources. How do you detect and prevent this?

Answers

1. **AWSCloudFormationStackSetAdministrationRole** in the admin account (trusts CloudFormation); **AWSCloudFormationStackSetExecutionRole** in each target (trusts the admin account).
2. **Self-managed.**
3. **Service-managed StackSet** targeting the Workloads OU with **auto-deployment enabled**.
4. Service-managed **never deploys to the management account**. Use a **self-managed** StackSet or a normal stack there.
5. `--call-as DELEGATED_ADMIN`.
6. To **avoid using the management account** day to day (least privilege).
7. **RetainStacksOnAccountRemoval**: false deletes the stack, true keeps it (no longer managed).
8. **No.** Run `create-stack-instances` for the OUs with the new Region.
9. DeploymentTargets with **AccountFilterType DIFFERENCE** and the sandbox account ID.
10. **FailureToleranceCount=0** and **RegionConcurrencyType SEQUENTIAL** (optionally RegionOrder).
11. **Managed execution Active=true** (queues or runs operations concurrently).
12. **IAM is global** and the role name is fixed; deploy IAM resources to **one Region**.
13. **Delete all stack instances** (or delete them with `--retain-stacks`).
14. `delete-stack-instances --retain-stacks`, then remove leftovers manually.
15. Detect with **StackSet drift detection**; prevent with an **SCP** denying changes to those resources and the StackSets execution roles.

Quick memory summary

- **StackSet** = one template to **accounts x Regions**; **stack instances** are pointers.
- **Self-managed**: you create **admin + execution roles**; any account (outside org, management account); **no auto-deploy**.
- **Service-managed**: **trusted access**, automatic roles, **OU targets**, **auto-deployment**, **delegated admin** (`--call-as DELEGATED_ADMIN`), **never the management account**.
- **Auto-deployment**: joining an OU adds stacks, leaving removes them (or keeps with RetainStacksOnAccountRemoval); **new Regions are manual**.
- **Rollout**: RegionOrder, SEQUENTIAL / PARALLEL, max concurrent, **failure tolerance per Region**, strict / soft, **managed execution**.